

1. Define static binding and dynamic binding.

[Explain your answers with examples for each one] (4 points)

Static Binding (Early Binding): Static binding happens when the association between a function call and the function code is made at compile time, meaning before the program runs. Since everything about the function is known in advance - its location, arguments, and behavior - this connection does not change while the program is running.

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3); // The call to add() is statically bound.
    printf("%d", result);
    return 0;
}
```

In this C program, when you call `add(5, 3)`, the compiler already knows where the function `add` is defined, what parameters it takes, and what type it returns. This is all resolved at compile time, hence static binding.

Dynamic Binding: Dynamic binding occurs when the function or method to be executed is determined at runtime. C does not support true dynamic binding as seen in object-oriented languages with polymorphism and method overriding. However, a form of runtime decision-making can be achieved using function pointers, which allow the program to decide which function to call during execution.

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

def make_animal_sound(animal):
    animal.sound() # Dynamic binding happens here.

a = Animal()
d = Dog()

make_animal_sound(a) # Prints "Some sound"
make_animal_sound(d) # Prints "Bark"
```

2. Define the subprogram protocol. (2 points)

The protocol of a subprogram is its parameter profile plus, if it is a function, its return type. In languages in which subprograms have types, those types are defined by the subprogram's protocol.

The protocol ensures that the subprogram operates correctly within its environment and that the calling and called environments are properly managed and isolated as necessary. This is critical for maintaining the integrity and reliability of the software.

3. How can you describe a variable? What is a variable? (1 point)

List and explain all the attributes of a variable (excl. lifetime and scope) (4 points)

List and explain all the attributes of a variable (incl. lifetime and scope) (6 points)

A variable is a symbolic name associated with a value and whose associated value may be changed during the execution of a program. Variables are fundamental in programming because they allow for the temporary storage and manipulation of data within a program.

- **Name:** The identifier used to access the variable in the program. It must be unique within its scope.
- **Type:** Defines the kind of data the variable can hold (e.g., integer, float, string). This affects the operations that can be performed on the variable and the amount of storage it occupies.
- **Value:** The actual data stored in the variable at any given time. The value of a variable can change as the program runs.
- **Address:** Is the machine memory address with which it is associated with.
- **Lifetime:** Is the time duration in which the variable is bounded to a specific memory location.
- **Scope:** The region of the program code where the variable can be accessed. Variables might be scoped to a single function, the entire program, or somewhere in between (like within a block of code).

4. Define the referencing environment for a static-scoped language (2 points)

In a static-scoped language, the referencing environment of a statement includes all local variables, and all visible variables declared in ancestor scopes.

This is determined at compile time, based on the program's lexical structure, not at runtime.

5. What does it mean when a variable is implicit heap dynamic? (3 points)

(Explain its advantages and disadvantages!)

Implicit heap-dynamic variables are bound to heap storage only when they are assigned values. In fact, all their attributes are bound every time they are assigned. This feature's common in languages with automatic memory management like Python and JS.

The advantage of such variables is that they have the highest degree of flexibility, allowing highly generic code to be written. One disadvantage of implicit heap-dynamic variables is the run-time overhead of maintaining all the dynamic attributes, which could include array subscript types and ranges, among others. Another disadvantage is the loss of some error detection by the compiler.

6. What are the design issues for arrays? (2 points)
[Explain your answers for C] (3 points)

- Subscript type – What types are legal?
- Subscript range binding – When are indices bound?
- Index range checking – Is it done at compile/runtime?
- Storage allocation – Static, stack, heap, etc.?
- Multidimensional support – Are rectangular/ragged allowed?
- Initialization – Can arrays be initialized when allocated?
- Slices – Are subarrays supported?

C-specific Explanation (3 points):

In C:

- "Arrays are homogeneous — elements must be the same type."
- "Subscript indices must be integers and are not range-checked."
- "Indexing always starts at 0, and bounds are fixed at compile time for static arrays."
- "C supports static, stack-dynamic, and heap-dynamic arrays (via `malloc`)."
- "Multidimensional arrays are allowed, but ragged arrays must be simulated with pointers."
- "Slices are not supported, though pointer arithmetic allows similar behavior."

7. What is unusual about C's multiple selection statement? (2 points)

In C, the **switch** statement is unusual because it **allows fall-through**: if no **break**; is used, execution continues into the next **case**. This can lead to unintended behaviour if not handled carefully. Also, **case** labels must be **constant expressions**, and C does not enforce implicit scoping for cases.

8. What are two common problems with pointers? (2 points)

Explain your answer! (2 points)

Dangling pointer: When an object with an incoming reference is deleted or deallocated without modifying the value of the pointer, leaving the pointer still pointing to the memory location of the deallocated memory.

Lost Heap-Dynamic Variable: An allocated Heap-Dynamic Variable that is no longer accessible to the user program, but still contains some useful data (often called "garbage").

9. What are the three semantic models of parameter passing?

Explain the Pass-by-value-result model in detail! (4 points)

Is parameter collision present in this mode?

Pass-by-Value:

The actual parameter's value is copied into the formal parameter.

Changes made to the formal parameter do not affect the actual argument.

Pass-by-Reference:

The formal parameter is a reference (alias) to the actual parameter.

Changes to the formal parameter directly affect the actual argument.

Pass-by-Value-Result (also called copy-in copy-out):

A hybrid model:

On call: The actual value is copied into the formal parameter (like pass-by-value).

On return: The final value of the formal parameter is copied back to the actual parameter.

Pass-by-Value-Result (Copy-In Copy-Out) – Explained ()

It behaves like pass-by-value during the call: arguments are evaluated and copied into formal parameters. But at the end of the function, the final value of the formal parameter is copied back to the actual parameter. Often used when the actual parameter is a variable, not an expression.

Side effect: If two actual parameters refer to the same variable, the order of copying out matters and can cause inconsistencies.

Parameter Collision: If the same variable is passed multiple times as an actual parameter, unexpected results can occur due to the order of updates when the subprogram terminates. For example, if a variable `x` is passed twice to a subprogram, changes made to the first instance might be overwritten by the assignment to the second instance.

I0. Define steps to the semantics of a call to a “simple” subprogram! (2 points)
Show an example!

The semantics of a simple subprogram call involve the following steps:

1. “Save the execution status of the caller (e.g., program counter).”
2. “Evaluate and pass parameters to the subprogram.”
3. “Pass the return address to allow resuming in the caller.”
4. “Transfer control to the subprogram.”
5. “After execution, return control and result (if any) to the caller.”

Example in C:

```
c
int add(int a, int b) {
    return a + b;
}
int main() {
    int result = add(2, 3); // subprogram call
    return 0;
}
```

I1. What is a record as a data type? (1 point)
Compare it with arrays! (2 points)

A **record** is a data type that stores a **fixed collection of fields**, each identified by a **name** and possibly having **different data types**.

Comparison with arrays:

- “Arrays store elements of the same type, accessed by index.”
- “Records store heterogeneous fields, accessed by name.”
- “Arrays are best for uniform data; records are ideal for representing entities like a person (name, age, ID).”

I2. What does it mean when a variable is stack-dynamic? (3 points)
(Explain its advantages and disadvantages!)

Stack-dynamic variables are those whose storage bindings are created when their declaration statements are elaborated, but whose types are statically bound.

Advantage: allows recursion; conserves storage.

Disadvantages: Overhead of allocation and deallocation possibly slower accesses (indirect addressing) Subprograms cannot be history-sensitive.

I3. Define the two fundamental kinds of subprograms! (2 points)
What are the similarities and differences between them? (3 points)

Function: A block of code that only runs when it is called.

Procedure: A collection of statements that define parameterized computations

Similarities: names can be called /accept input parameters.

Differences:

Function: have return value; are used in expressions or assigned to variables.

Procedure: no return value; are invoked for their execution.

I4. What is an associative array? (2 points)
[Explain your answers for a chosen language!] (2 points)

An **associative array** is a collection of **key-value pairs** where keys are unique and used to access values. Unlike normal arrays that use integer indices, associative arrays use keys like strings.

In Python, the `dict` type is an associative array:

```
python
user = {"name": "Alice", "age": 25}
print(user["name"]) # Outputs: Alice
```

15. What could be a problem with nested selection statement in C based language? (2 points)

In C-based languages, nested selection statements can lead to the **dangling else problem**, where it is ambiguous which **if** an **else** matches.

C always associates **else** with the **nearest unmatched if**, which may cause logic errors.

To avoid this, use braces **{ }** to clarify the intended control structure.

16. What are special words? (1 point)

Explain your answer with its subtypes and examples! (3 points)

Special words are predefined words in a language that have syntactic meaning and **cannot be used as identifiers**. They help define the **structure and control** of programs.

Subtypes include:

1. "Control flow keywords – **if**, **else**, **switch**, **while**"
2. "Type declarations – **int**, **float**, **char**, **class**"
3. "Visibility modifiers – **public**, **private**, **protected** (Java, C++)"
4. "Storage specifiers – **static**, **extern**, **volatile**"

These keywords ensure code is interpreted consistently by compilers and help enforce language rules.

17. Explain the Pass-by-result model in detail with examples.

Discuss the well-known potential problem with it! (4 points)

In **pass-by-result**, no initial value is passed to the subprogram. The final value of the formal parameter is **copied back** to the actual parameter only after the subprogram completes.

Example:

pseudocode

Copy

Edit

```
procedure increment(x: result int)
  x := x + 1
```

The actual parameter gets updated only after the function ends.

Known problem: If the same variable is passed to multiple **result** parameters, the **order of copy-back matters**, causing **unpredictable final values** (e.g., `update(a, a)` → result could be 10 or 20). This is known as the **aliasing issue**.

Let me know if you're ready for Question 18!

18. Explain the Pass-by-reference model in detail with examples.

Discuss the well-known potential problem with it! (4 points)

The pass-by-reference model in programming involves passing the address (reference) of a variable rather than its actual value to a function. This means that any modifications made to the parameter inside the function affect the original argument used to call the function.

```
int func(int &a, int &b){
  ...
}
int main(){
  int x, y, z;
  z = func(x, y)
}
```

Potential Problems with Pass-by-Reference: One well-known potential problem with the pass-by-reference model is unintended side effects. Since the function operates on the original data, any changes made to the parameter within the function are reflected in that original variable. This can lead to bugs if the programmer does not expect a function to modify its arguments.

19. Which produces faster program execution: a compiler or a pure interpreter? (1 point)

[Explain your answer!] (3 points)

A compiler produces faster program execution compared to a pure interpreter, mainly because it translates the entire source code of a program into machine code before the program is even run. This translation process, though potentially time-consuming initially, results in a directly executable binary file that is highly optimized for the target hardware.

In contrast, a pure interpreter reads, analyses, and executes the code line-by-line at runtime, which introduces a significant amount of overhead for each instruction. This constant interpretation cycle can substantially slow down program execution because each instruction must be parsed and converted into machine operations every time it is executed.

20. Define the meaning of syntax and semantics. (2 points)

Syntax in programming refers to the rules that define how to correctly arrange symbols and commands in code, much like grammar in language.

Semantics, on the other hand, concerns the meanings of these syntactic elements, determining what the code does when it runs. Essentially, syntax is about structure, and semantics is about meaning.

21. What is the definition of block? (2 points)

In programming, particularly in languages like C, a **block** is a group of zero or more statements enclosed in curly braces `{}`. It defines a local scope. Variables declared within a block are local to that block; they can only be accessed from within the block and cease to exist when the block is exited. Blocks can be used to create complex control structures by enclosing multiple statements that should be executed together, such as the body of a loop, a conditional branch, or a function.

22. What does it mean when a variable is static? (4 points) (Explain its advantages and disadvantages!)

In programming, when a variable is declared as static, it means that the variable retains its value between function calls and is not reinitialized each time the function in which it is declared is invoked. This behavior contrasts with local variables that are typically initialized each time a function is called.

Advantages:

- Persistence of Value: Useful for storing state information within a function without needing global variables.
- Controlled Scope: Limited to the block they are defined in, preventing access outside their intended context.
- One-time Initialization: They are initialized only once, improving performance.

Disadvantages:

- Memory Usage: Remain in memory throughout the program's lifetime, potentially using more memory.
- Concurrency Issues: Can lead to data corruption in multithreaded applications unless properly synchronized.
- Hidden State: Makes function behavior less predictable due to dependencies on retained state from previous calls.

23. What are the design issues for multiple-selection statements? (2 points) [Explain your answers for C specifically] (3 points)

Expression Type:

C: Only integral types (e.g., **int**, **char**) are allowed for the controlling expression, limiting flexibility compared to languages that allow other types.

Case Labels:

C: Case labels must be constant integral expressions, preventing dynamic case evaluations.

Default Behaviour:

C: Including a default case is optional but helps handle unexpected values.

Action Scope:

C: Without explicit break statements, C allows "fall-through" from one case to another, which can lead to bugs if not managed carefully.

Range of Values:

C: C does not support specifying ranges in case labels; each case must be explicitly defined.

Nested Selections:

C: switch statements can be nested, but this can complicate readability and maintenance.

24. How does C support relational and Boolean expressions? (2 points)

For C it doesn't have built in Boolean expressions, but it supports it by integer values, relation operators and logical operators. All operands with nonzero values are considered true, and zero is considered false.

Relation operators in c: <, ==, >= . Logical operators in c: &&, ||, ! .

25. In what way is C's for statement more flexible than that of many other languages?
[Show examples!] (3 points)

C's for is more flexible because each of the expressions can comprise multiple statements, which in turn allow multiple loop vars that can be of any type.

Example:

```
1. for (i = 0; i < 5; i++) {  
2.  printf("Hello, world!\n");}
```

This will print Hello, world 5 times and stop.

26. What is an array as a data type? (1 point)
(Compare it with records!) (2 points)

An array is a collection of elements, all of the same type, that are accessed by indexing. Arrays are static in size, meaning that the number of elements they can hold is fixed when they are created. In the C programming language, arrays are declared by specifying the type of elements they will contain and the number of elements they will hold.

Comparison with Records: In contrast, a record (known as a struct in C) is a composite data type that groups together elements of potentially different types under a single name. Records are used to construct more complex data structures where components can have varied data types.

27. What does it mean when a variable is heap-dynamic? (3 points)
(Explain its advantages and disadvantages!)

A **heap-dynamic variable** is a variable that is **allocated and deallocated explicitly at runtime** using programmer-defined operations. These variables reside in the **heap memory**, and their size and lifetime are determined during program execution. In C, this is typically done using malloc or free.

Advantages:

- Allows **flexible memory allocation** during execution.
- Supports **dynamic data structures** like linked lists and trees.

Disadvantages:

- Requires **manual memory management**, increasing the risk of memory leaks and dangling pointers.
- Heap access is generally **slower** than stack access and can lead to **fragmentation**.

28. What is a union type? (2 points)
[Explain your answer for a chosen language!] (2 points)

Union Type

In C, a union type allows one portion of memory to be accessed as different data types. It is a structure in which all members share the same memory location. This is useful when you need to store different types of data in the same memory location without creating separate storage for each type.

```
union Data {  
    int i;  
    float f;  
    char str[20];  
};
```

29. What is a reserved word? (1 point)
Explain your answer with examples and indicate the difference from keywords! (3 points)

Reserved words are terms that are part of the vocabulary of the language and are reserved for a particular syntactic or semantic purpose; they cannot be redefined or used as identifiers (like variable names). In C, examples include int, return, and struct.

Difference from Keywords: While all reserved words are keywords, not all keywords might be reserved words in all contexts. Keywords are specifically used to identify the constructs of the programming language. In some languages, certain keywords can be used as identifiers if they are not currently serving their keyword purpose in a particular context. However, in C, the terms reserved word and keyword are often used interchangeably.

30. Define the concept of address binding for static and dynamic cases! (4 points)
[Explain your answer with examples for each one!]

Address binding is the process of associating a program's variables or instructions with specific memory addresses. It can occur at different stages of program execution.

Static Address Binding: it occurs at compile time (or load time). The address of the variable is determined before the program runs and remains fixed throughout execution.

Example (C):

```
int x = 5;
```

Dynamic Address Binding: it occurs at runtime. The memory for the variable is allocated during program execution, typically on the heap or stack, depending on how the variable is created.

Example (C):

```
int *p = malloc(sizeof(int));
```

31. How to create a new (static) scope inside a subprogram? (1 point)
Show an example! (1 point)

A **new static scope** can be created inside a subprogram by placing a **nested block** (compound statement) within the subprogram body.

In C, this is done using { ... }. Each block defines a **new static scope**, where variables can be declared and are **visible only within that block**.

32. Why are C and C++ not strongly typed? (3 points)

Unions are potentially unsafe constructs in some languages. They are one of the reasons why C and C++ are not strongly typed: These languages do not allow type checking of references to their unions. You can cast between incompatible pointer types (e.g., from int* to float*, or even void*), which breaks strict type safety.

33. What is a compatible type? (1 point)

A compatible type is one that either is legal for the operator or is allowed under language rules to be implicitly converted by compiler-generated code (or the interpreter) to a legal type.

34. What is a referencing environment of a statement? (3 points)
Explain it with one sentence for static and dynamic cases!

The referencing environment of a statement is the set of all bindings between identifiers and their corresponding entities (like variables and functions) that are visible and can be referenced from that statement. This environment is determined by the scope rules of the language.

35. What is a pointer type? (1 point)

[Discuss it! (heap, anonymous var, dereferencing, operators)] (2 points)

A **pointer type** is a type in which variables hold **memory addresses** of other variables rather than actual values. Pointers may also hold a special value like nil (or NULL in C) to represent that they are not currently pointing to a valid memory location.

Heap-dynamic variables: Pointers are used to reference variables allocated in the heap at runtime using dynamic memory management (e.g., malloc() in C).

Anonymous variables: Heap-allocated variables often have no identifier and can only be accessed via pointers; these are called anonymous variables.

Dereferencing: Pointers enable indirect addressing by using dereferencing (*) to access or modify the value at the referenced address.

Operators: In languages like C and C++, pointers use:

* to dereference,

& to get the address,

-> to access members of a struct via pointer.

36. What is a coroutine? (1 point)

Explain your answer with details! (3 points)

A coroutine is a generalization of a subroutine. Unlike subroutines, coroutines allow multiple entry points for suspending and resuming execution at certain locations, and can maintain their state (local variables, instruction pointer, etc.) between activations. This makes them useful for implementing cooperative multitasking within a single program.

37. Define the concept of type binding for static and dynamic cases! (4 points)
[Explain your answer with examples for each one!]

Type binding is the process of associating a variable with a type, either at compile time (static binding) or at runtime (dynamic binding).

Static Type Binding:

- Occurs at compile time; the type is fixed before the program runs.
- Example (Explicit): In C++, `int x;` explicitly sets `x` as an integer.
- Example (Implicit): In C#, `var x = 10;` implicitly infers `x` as an integer.

Dynamic Type Binding:

- Occurs at runtime; the type can change as the program executes.
- Example: In Python, `x = 10` can be followed by `x = "hello"`, changing `x` from an integer to a string.

Static binding enhances type safety and execution efficiency, while dynamic binding offers flexibility by allowing types to change during execution.

38. Explain the three reasons why lexical analysis is separated from syntax analysis. (4 points)
[Explain your answer with examples for each one]

Simplification of Design: Lexical analysis involves scanning the code to convert sequences of characters into tokens, such as keywords, identifiers, operators, etc. By separating these concerns, the syntax analyzer can operate with a clean set of well-defined tokens, simplifying the parsing process.

Example: A lexer converts character sequences into tokens like `int`, `if`, etc., which the parser then uses to construct syntactic structures without worrying about character-level details.

Efficiency: Separating lexical analysis allows optimizations that can make compilation faster. The lexical analyzer can be optimized independently to scan and tokenize the input stream efficiently.

Example: Implementing a highly optimized state machine for the lexer that rapidly processes input characters.

Error Handling: Errors in lexical analysis and syntax analysis can be managed more effectively when handled separately. The lexer can focus on character and token level issues, while the parser can handle errors in syntax structures.

Example: The lexer can easily catch forbidden characters or malformed tokens, and the parser can manage grammatical errors in code.

39. What is a static ancestor of a subprogram?

What is a dynamic ancestor of a subprogram? (2 points)

Static Ancestor: A static ancestor of a subprogram is any subprogram that textually encloses another subprogram in the program's source code. This relationship does not change during execution and is determined by the lexical structure of the program.

Dynamic Ancestor: A dynamic ancestor of a subprogram is a subprogram from which another subprogram is called during execution. This reflects the call stack's state at any point in time during the program's execution.

40. What is an overloaded operator? (2 points)

An overloaded operator is an operator that has more than one meaning depending on its context, particularly the types of its operands. For example, in C++, the `+` operator is used for both numeric addition and string concatenation, depending on the operand types.

41. What is a generic subprogram? (2 points)

A generic subprogram is a type of subprogram that is written with type parameters, allowing it to operate on a variety of data types without being rewritten for each type. This supports writing more general, reusable code. For example, a generic sort function can sort items of any type for which comparison is defined.

42. How does short-circuit expression evaluation work? (2 points)

Short-circuit evaluation means that in logical expressions (typically involving `and` and `or`), the second operand is evaluated only if the first operand does not determine the result. For instance, in the expression `a && b`, `b` is not evaluated if `a` is false since the whole expression will be false regardless.

43. Discuss dynamic scoping in detail! (3 points) (binding names, nesting, hiding, ...)

Dynamic scoping is a type of name binding where the referencing environment of a statement is determined by the **runtime calling sequence** rather than the program's lexical structure.

Binding of names:

In dynamic scoping, when a variable is referenced, the language system **searches the call stack** (active subprograms) from the current function upward to find the **most recent active declaration** of that name.

Nesting and hiding:

Dynamic scoping does **not rely on lexical (static) nesting**. Instead, **variable visibility is determined at runtime**.

Variables declared in inner subprograms may **override** (hide) those in calling subprograms, but the hiding is **temporal** rather than structural.

What are the advantages and disadvantages of dynamic type binding?

Advantages:

Flexibility: Allows a variable to be bound to objects of different types during the execution of a program, which can make the program more flexible and easier to write.

Support for Polymorphism: Facilitates programming in a polymorphic style, enabling a single function to process objects of different types.

Disadvantages:

Performance: Can lead to slower execution because type compatibility checks must be performed at runtime.

Safety: Increases the risk of runtime type errors, as errors that could be caught at compile-time in statically typed languages are only caught at execution.

44. Define static scope! Show an example for nested scope in the C language!

Static scope, also known as lexical scope, is a feature in some programming languages where the visibility and accessibility of a variable is determined by its position within the textual code. A variable's scope is defined by its block structure at compile time, meaning that where the variables are declared in the code determines where they can be used.

```
#include <stdio.h>

void example() {
    int x = 10; // Outer variable in outer scope

    {
        int y = 5; // Inner variable in nested scope
        printf("%d\n", x); // x is accessible here due to static scoping
    }

    // printf("%d", y); // ✗ Error: y is not visible here
}
```

45. Using the following JavaScript code, explain the scope for “x” variables! (2 points)

```
function O_big_sub() {
    function sub1() {
        var x = 7;
        sub2();
    }
    function sub2() {
        var y = x;
    }

    var x = 3;
    sub1();
}
```

- `x = 7` in `sub1()` has **local scope** only inside `sub1()` and **does not affect** `sub2()` .
- `x = 3` in `0_big_sub()` has **function-level scope** and is the one used by `sub2()` .

44. Define scope and lifetime! (2 points)

Explain C's scoping and lifetime for all count variables in the code fragment: (3 points)

```
void sub () {
    int count;
    ...
    while( ... ) {
        int count;
        count ++
        ...
    }
    ...
}
void printhead () {
    int sum;
    ...
    sub ();
}
```

Definition:

- **Scope** is the region of a program where a variable is **visible and accessible**. In C, scope is determined by **block structure** (`{ ... }`).
- **Lifetime** refers to the **duration during program execution** when a variable **exists in memory**. Local variables in C have **automatic storage duration**, meaning their lifetime begins when the block is entered and ends when the block is exited.

♦ **Explanation of `count` variables:**

(1) **Outer `count` (in `sub()`):**

- **Scope:** From its declaration to the end of the `sub()` function.
- **Lifetime:** From function entry to function exit (automatic duration).
- **Note:** Hidden inside the `while` block due to re-declaration.

(2) **Inner `count` (inside `while` loop):**

- **Scope:** Limited to the body of the `while` loop.
- **Lifetime:** Begins at each loop iteration and ends at the end of that iteration.
- **Note:** Shadows the outer `count` within the loop.